A GHCi session, step by step

Each step appears on its own overlay:

```
ghci> let xs = [1 .. 10]
```

A GHCi session, step by step

Each step appears on its own overlay:

```
ghci> let xs = [1 .. 10]
ghci> sum xs
55
```

A GHCi session, step by step

Each step appears on its own overlay:

```
ghci> let xs = [1 .. 10]
ghci> sum xs
55
ghci> filter even xs
[2,4,6,8,10]
```

A GHCi session, step by step

Each step appears on its own overlay:

```
ghci> let xs = [1 .. 10]
ghci> sum xs
55
ghci> filter even xs
[2,4,6,8,10]
ghci> :t foldr
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
```

A GHCi session, step by step

Each step appears on its own overlay:

```
ghci> let xs = [1 .. 10]
ghci> sum xs
55
ghci> filter even xs
[2,4,6,8,10]
ghci> :t foldr
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
ghci> foldr (\x acc -> x + acc) 0 xs
55
```

Types from GHC

```haskell
fib :: Integer -> Integer
fib n = if n < 2 then n else fib (n - 1) + fib (n - 2)
```

The type of `zipWith` is `(a -> b -> c) -> [a] -> [b] -> [c]` and `take 5 [2, 4 ..]` is `[2,4,6,8,10]`.